

1 Experimental Setup

This section specifies, in full, the data, the knowledge-graph construction, the inference models, the online evaluation protocol, the metrics, the baselines, and the reproducibility conditions under which every result in this paper was produced. Two principles govern the whole design and are worth stating up front because they recur throughout. **First, everything is *just-in-time (JIT)* and *fully online*:** the graph grows one commit at a time in true chronological order, and a commit is always predicted from a model and a graph state built *only* from strictly earlier commits — there is no random split, no look-ahead, and no statistic is ever fit on data that includes or postdates the commit under test. **Second, the study is designed as a controlled comparison:** when we ask which code representation or which inference method is best, exactly one factor is varied at a time while the protocol, the classifier family, the warm-up, and the block schedule are held fixed, so any measured difference is attributable to that factor and not to a confound.

1.1 Subject program, commit scope, and labels

Project. All experiments are conducted on `apache/groovy`, a mature, long-lived Java project (the Groovy programming language and its compiler, runtime, and tooling). We work from a full local `git` clone, so the raw material for every commit — its message, its parent(s), and the exact byte content of every file before and after the change — is available offline and deterministic.

Labelled commit scope. The study set is the 8,059 **commits** of `apache/groovy` that appear in the **ApacheJIT** dataset (`data/apachejit/projects/apache_groovy.csv`), spanning **2003–2019**. ApacheJIT provides, for each commit, a *buggy/benign* label derived by an SZZ-style procedure (a commit is buggy if a later fix commit points back to it) and the twelve standard JIT change metrics. These 8,059 commits are the only ones we treat as the labelled study set and the only ones grown into the AST/structural and semantic layers. The surrounding history is retained in the database as **Commit** nodes for context (the full clone has 16,887 commits), but only the ApacheJIT commits are flagged `in_jit=true` and are the units of prediction.

Class balance and its consequences. The labelled set is imbalanced: 1,614 **buggy** / 6,445 **benign**, i.e. a $\approx 20\%$ positive rate overall. Crucially, the positive rate is *not stationary*: it rises from $\approx 6\%$ in 2003 to a peak of $\approx 39\%$ in 2012 and then falls back to $\approx 6\%$ by 2019 (measured directly from the labels). This pronounced *concept drift* is the single most important property of the data for our purposes: it is why a random or single-split evaluation is inappropriate, why we adopt a time-aware online protocol, and why the tail of every online trajectory declines (the achievable precision/recall shrink as positives become rare). We return to this when defining the protocol (§1.5) and the metrics (§1.6).

Base snapshot. The structural layers are seeded once at a fixed *base* commit — `408b29851d7bbe4d343340832` the first substantial Java commit — whose 120 `.java` files ($\approx 46k$ AST nodes) constitute the initial code structure onto which all subsequent per-commit changes are laid. The base is a property of the *code*, not of the labelled set; it merely fixes the starting state of the evolving graph.

1.2 Knowledge-graph construction and online growth

The knowledge graph is a *heterogeneous information network* (HIN): typed nodes and typed edges in one store (Neo4j). It fuses three families of evidence that the defect-prediction literature usually treats separately — *process* evidence (who changed what, when, and how often), *code-structure* evidence (the syntactic/semantic shape of the change and its surroundings), and *textual* evidence (the natural-language and lexical content of the change). The design rationale for a graph (rather

than a flat feature table) is that anchoring each commit’s change to *shared, persistent* code entities is what makes relational and neighbourhood reasoning across the entire history possible.

Core (process / relational) layer. The Core layer is the classical commit graph and is present in *every* experimental configuration. Its node types are **Project**, **Branch**, **Developer**, **Commit**, **File**, and **Issue**; its edges encode authorship (Commit^{AUTHORED_BY}→Developer), the history spine (Commit^{PARENT_OF}→Commit), file changes (ADDED/MODIFIED/DELETED/RENAMED_FROM/RENAMED_TO between Commit and File), and issue links (Commit^{FIXES_ISSUE}→Issue). Every Commit also carries the twelve ApacheJIT change metrics (size **la**, **ld**; dispersion **nf**, **nd**, **ns**, **ent**; history and experience **age**, **ndev**, **nuc**, **aexp**, **arexp**, **asexp**) and its **author_ts** (the UNIX timestamp that defines online arrival order), **buggy** label, and **jit_year**.

Structural (code) layers as *delta* graphs. On top of the Core, we materialise the code structure of every changed `.java` file. The key modelling decision — and a central contribution — is that a commit is not attached to a per-commit snapshot blob; it is attached, by four typed *delta* edges (ADDS, REMOVES, UPDATES, MOVES), directly to the specific code nodes it changed, laid on top of a single evolving structure per file. Concretely, for each tracked file the graph holds one *current* structure; when a commit touches the file we (i) parse the before- and after-states with the *same* deterministic builder, (ii) diff them, (iii) write the delta edges from the Commit to the affected nodes, and (iv) evolve the stored structure so the next commit diffs against the correct state. Node identity is threaded across versions by a deterministic, per-file parse-local-id map, so the “after” map of commit *N* is byte-identical to the “before” map of the next commit that touches the file when the intervening source is unchanged. Removed nodes are *retained* and flagged (**alive=false**, **removed_by**), so the complete change history is queryable while the live structure always reflects the current code. This makes storage grow with the *amount of change*, not with history-length × file-size, and it is what makes the whole graph online-extensible.

We instantiate this delta-graph scheme for **five** structural representations of increasing coarseness, all built per changed `.java` file and attached at the File level:

- **AST** — the abstract syntax tree; one node per syntactic construct (declaration, statement, expression, operator, literal, and each identifier/operator leaf). Finest granularity (136 node types).
- **CFG** — the intra-procedural control-flow graph; one node per statement or control construct (18 node types).
- **DFG** — the def–use data-flow graph over local variables; one node per definition/use site (7 node types).
- **PDG** — the program-dependence graph; the CFG’s statement vertices plus control-dependence and data-dependence *edges* (no new nodes; 18 node types). We label it PDG rather than CPG because we deliberately keep dependence at statement granularity and do *not* embed the AST subtree under each statement; a canonical code property graph would contain the AST as a subgraph and is out of scope.
- **Token-seq** — a deliberately weak flat statement sequence (statement nodes chained by NEXT, no control or data structure); a lower-bound rival used only in the representation comparison.

Table 1 reports the materialised scale of each layer. The $\approx 10\times$ gap between AST (3.70M nodes) and the others (0.26–0.47M) is a pure *granularity* effect: a statement such as `int x = foo(a,b)+1;` is one node in CFG/PDG but a dozen-plus nodes in the AST. CFG and PDG

Table 1: Materialised scale of each structural layer and its per-commit change-token stream (Groovy, 8,059 commits). “Token types” counts the distinct `{edge}:{node-type}` change-tokens; “Commits w/ tok.” is the number of labelled commits that emit at least one token in that layer.

Layer	Nodes	Delta-edges	Node types	Token types	Commits w/ tok.
AST	3,699,439	3,614,922	136	314	7,523
CFG	372,488	408,301	18	53	6,194
DFG	469,662	648,553	7	27	6,066
PDG	372,488	411,869	18	63	6,261
Token-seq	257,708	293,287	16	47	6,011

have identical node counts because the PDG adds only dependence edges to the CFG’s vertex set.

The differ, and why matching is fair across layers. The AST layer is diffed as a *tree* by bottom-up subtree hashing (identical subtrees in the before and after trees are matched greedily, preferring equal source positions; unmatched before-nodes are deletions, unmatched after-nodes insertions, matched-but-relabelled nodes updates, reparented nodes moves). The four non-tree layers (CFG/DFG/PDG/Token-seq) are general typed graphs and are diffed by a single *node-identity-lite* matcher applied *identically* to each: matching is scoped within a method (`class::name`, stable across commits) and keyed by *ordinal* (the k -th node of a given type in a method maps to the k -th such node in the other version), which is robust to line shifts. Using one shared differ for the alternatives is the fairness guarantee for the representation comparison: only the subgraph varies, never the difference model. We disclose that the AST uses a tree differ while the others use the ordinal matcher — a deliberate specialisation (trees vs. general graphs), not a hidden confound.

Semantic-text layer (CSTG). The third first-class layer is the **Commit Semantic-Text Graph (CSTG)**, which models the *textual* description of a change (commit message + diff text) as a graph grounded in code. Its node types are **Term** (a typed vocabulary term, with $\text{kind} \in \{\text{code-identifier}, \text{bug-lexicon}, \text{action}, \text{error-type}, \text{nl}\}$) and **Intent** (the change-intent class: fix, feat, refactor, ...); its edges are $\text{Commit} \xrightarrow{\text{MENTIONS (w=TW-IDF)}} \text{Term}$ (top terms per commit, weighted by TextRank centrality in the commit’s word-graph $\times$ IDF), $\text{Term} \xrightarrow{\text{COOCCURS (npmi)}} \text{Term}$ (the global normalised-pointwise-mutual-information term graph), $\text{Term} \xrightarrow{\text{GROUNDS_IN}} \text{ASTNode}$ (a code term linked to the alive identifier leaf it names), and $\text{Commit} \xrightarrow{\text{HAS_INTENT}} \text{Intent}$. On Groovy the CSTG comprises 20,377 Term nodes, 188,435 MENTIONS, 120,340 COOCCURS, 19,315 GROUNDS_IN, and 8,059 HAS_INTENT. Theoretically the layer rests on graph-of-words / TW-IDF (Rousseau & Vazirgiannis), the NPMI corpus term graph (cf. TextGCN), defect-semantic typing, and term-risk propagation over the NPMI graph. Every CSTG statistic that enters prediction is grown online, past-only (see §1.5); the static term graph and term-risk values written to Neo4j are a queryable/visualisation snapshot and are *not* read by the online predictor.

Data-quality guarantees. The finalised graph passes a standing integrity audit: no file has more than one structure root, removed nodes always carry a REMOVES edge, and the online-evolved structure reproduces a fresh ground-truth parse of the current file state with precision = recall = 1.0 on the validated sample. A large *ghost-AST* inflation (≈ 1.58 M nodes left by package-restructuring renames outside the labelled set) was identified and corrected, so the reported node counts reflect the clean final state. Known, documented limitations that a reviewer should be aware of: labels are SZZ-derived and carry the usual mining noise; non-.java files are not

modelled at the code level; and a git rename is currently treated as an add-of-new-path in the delta layer.

1.3 Inference models

The target hardware has **no GPU**; every method is therefore deliberately CPU-only and **GNN-free**, both to be deployable and to constitute strong, honest baselines that any future GNN must beat. The final methodology fixes **five canonical graph-inference paradigms**, chosen because they map one-to-one onto the recognised families of graph and knowledge-graph inference, each reads a *different, complementary* facet of the structure, and — crucially — each is a graph *algorithm* that runs unchanged on *any* graph in the family, so all five are valid rows across every column of the representation study:

RN — Relational neighbour (wvRN). Collective classification: a commit’s score is the label-weighted hub-overlap with past commits (one hop through the shared file/developer/term hubs). Complexity $O(\text{nnz})$ of a sparse block matmul.

PPR — Personalised PageRank (random walk with restart). The bipartite commit–hub graph is seeded from past buggy vs. benign commits and a commit is scored by its relative proximity to each class. Complexity $O(\text{iters} \cdot |E|)$ per block, run once per class seed set.

LP — Label propagation. Spreading activation of past labels through the hubs a few steps, clamped on known past labels. Complexity $O(\text{iters} \cdot |E|)$.

DW — DeepWalk-style embedding. A random-walk / matrix-factorisation node embedding (PPMI of the commit–hub co-occurrence, truncated-SVD, the Levy–Goldberg form) with a logistic head; marginals from past rows only.

KGE — Knowledge-graph embedding (DistMult). A compact bilinear embedding over typed (commit, rel, hub) triples restricted to past commits, with commits folded in from their hubs’ learned embeddings and a logistic head.

The embedding methods (DW, KGE) are refit on the expanding past window every **5 blocks**; RN, PPR, and LP are recomputed each block from the strictly-past state. Embedding dimensions are 64 (DW) and 32 (KGE). All linear heads are balanced logistic regressions.

Two non-graph channels used only in fusion. Two additional signals are *not* admissible as rows of the representation study (they are not graph algorithms) but are evaluated as fusion channels:

- **M — JIT metrics:** a prequential logistic model over the twelve ApacheJIT change metrics; the classical JIT-defect predictor and our metric-only reference channel.
- **G — CSTG feature channel:** a prequential logistic model over the CSTG’s per-commit features — the graph-of-words (TextRank-weighted, add/remove polarity-tagged) term stream, the NPMI-propagated past-only term-risk prior, a 5-dimensional typed-mass vector over the defect lexicon, and the intent/consistency features — enriched with an online recency / bug-cache block (per-file and per-developer time-since-last-buggy touch, recently-buggy touched-file count, re-fix flag, commit velocity), each computed from strictly-past state.

The commit-*message* text channel (X) that appeared in earlier iterations is **removed entirely** from the final methodology (it is a strict subset of G and adds nothing over it); it is disabled throughout and reported only as a historical step.

Fusion. Method and channel scores are combined by *prequential logistic-regression stacking*: at each block a balanced logistic regression is fit on the strictly-past predicted probabilities of the constituent models and applied to the current block, refit on the expanding window. The *deployed* model is selected by an explicit load-vs-performance rule (§1.7) and is the purely graph-native $F+G = \mathbf{RN+PPR+CSTG}$, i.e. the two-method graph fusion $F = \mathbf{RN+PPR}$ augmented with the CSTG semantic channel G , *without* the non-graph JIT metrics M .

1.4 The two research questions and the graph family

The experiments answer two questions with one design: **(RQ1, which representation?)** does a richer code/semantic structure improve commit-level prediction, and which structural representation is best? **(RQ2, which inference method?)** among typical graph-inference paradigms, which best exploits the graph?

Both are answered by a single family of graphs of increasing richness, all sharing the Core relational layer, evaluated with the fixed five methods: *Core*; *Core+AST*; *Core+CFG*; *Core+DFG*; *Core+PDG*; and the deployed final *Core+AST+CSTG*. Reading a row (a method across graphs) answers RQ1 for that method; reading a column (all methods on one graph) answers RQ2 for that graph. A structural layer influences prediction through exactly one channel — the per-commit change-tokens `{edge}:{type}` that become typed hubs the walks and embeddings traverse — so swapping the representation means swapping the hub stream while everything else is held fixed. On the final graph, the hubs additionally include CSTG Term nodes via MENTIONS, so the same five methods traverse the semantic signal too. (Token-seq is included in the representation *ablation* but is not part of the final six-graph family, being a deliberately weak rival.)

1.5 Online evaluation protocol

Prequential, predict-then-grow. Commits are streamed in chronological order (`author_ts`). The protocol is *prequential*: for each commit we (i) predict its label from a model and graph state built only from strictly earlier commits, (ii) reveal the true label, (iii) grow the graph state (add the commit’s delta edges, term hubs, recency counters, co-occurrence counts), and (iv) periodically re-learn on the expanded window. This exactly reproduces the information available at deployment time and structurally precludes look-ahead leakage.

Warm-up, blocks, and refit schedule. The first 40% of the stream ($W = \lfloor 0.40 \times 8,059 \rfloor = 3,223$ commits) is a warm-up used only to initialise models and is *not* scored; the remaining 4,836 commits are the evaluation stream. Predictions are made per commit but models are refreshed on a **block** granularity of 200 commits (the expanding-window retraining that lets the models adapt to concept drift). This warm-up/block schedule, the classifier family, and all hyper-parameters are held *identical* across every representation, method, and fusion so that the only independent variable in each comparison is the one under study.

Leakage controls (stated explicitly). All scalars, vectorisers, IDF weights, NPMI graphs, term-risk priors, embedding bases, and stacking regressions are fit on past data only; PPR class seeds never include the commit under test; the graph-of-words weighting and recency lookups are commit-local; and the `HashingVectorizer` used for the CSTG text stream is stateless (a fixed hash with no learned vocabulary), so it cannot transport information from a future commit into an earlier row. The single-split controlled comparison used against the external text baseline (§1.8) is clearly separated from the deployed online path and never mixed with it.

Online operating point. Because the task is imbalanced and drifting, the hard-decision threshold is itself tuned *online*: it is re-tuned on the past-only predictions every 150 commits to maximise buggy-F1 on already-seen data, giving a leakage-free deployment operating point

at which all threshold-based metrics are read. Rolling trajectories are computed over a trailing window of $\text{ROLL} = 800$ commits (a visualisation smoothing only — it never enters the model or the reported scalars, which are computed once over the entire evaluation stream).

1.6 Evaluation metrics

Because the positive class is rare and drifting, no single metric suffices; we report a **seven-metric** suite plus two effort-aware measures, and treat the threshold-free and class-balanced metrics as primary.

- **Threshold-free ranking:** *ROC-AUC* (a.k.a. AUC) and, where reported, *PR-AUC* (area under precision–recall; its random baseline equals the positive rate, which is why absolute PR-AUC falls in the low-defect tail even for a good model).
- **Operating-point metrics at the online-tuned threshold:** *Precision*, *Recall* (buggy-class sensitivity), *Buggy-F1* (F1 of the positive class), *Macro-F1* (unweighted mean of both classes’ F1 — robust to imbalance), *G-Mean* ($\sqrt{\text{TPR} \cdot \text{TNR}}$), and *MCC* (Matthews correlation — the single most reliable summary under imbalance).
- **Effort-aware (deployment triage) metrics:** normalised P_{opt} (Kamei et al.: how close the model’s inspection order is to the ideal buggy-first, least-effort-first order, normalised so 0.5 is random and 1.0 optimal) and *ACC@20%LOC* (recall achieved when only the top 20% of churned lines are inspected). Effort is measured as **1a+1d**.

The two *class-balanced* metrics — Macro-F1 and G-Mean — are the ones we prioritise for model selection, precisely because they do not let a model win by exploiting the majority class in the low-defect regime. We emphasise that, given the concept drift, the meaningful reading of an online trajectory is *relative* (which method/graph stays highest) rather than the absolute end-of-stream value, since the falling positive rate depresses every method’s tail together.

1.7 Fusion selection rule

The deployed fusion is not hand-picked. We evaluate *all* $2^5 - 1 = 31$ combinations of the five methods on the final graph, score each on the full seven-metric suite, and select the smallest combination whose balanced score $(\text{Macro-F1} + \text{G-Mean})/2$ is within a fixed tolerance (0.005) of the best — a transparent trade-off of *load* (fewer methods, cheaper online) against *performance*. This selects $F = \text{RN} + \text{PPR}$. We then test, on top of F , whether the explicit CSTG channel G and/or the non-graph JIT metrics M add anything: G helps markedly and M does not (adding M alone hurts), so the deployed model is $F + G = \text{RN} + \text{PPR} + \text{CSTG}$. Reporting the full combinatorial ablation, and choosing by an explicit rule, is what makes the deployed model defensible rather than cherry-picked.

1.8 Baselines and comparison protocols

We compare against baselines at two levels. (i) **Within-project JIT baselines:** logistic regression, random forest, and gradient boosting over the twelve ApacheJIT metrics — the standard JIT-defect predictor — evaluated under the identical online protocol, plus a naive prior-rate predictor. (ii) **A strong external text baseline:** a 768-dimensional diff-text TF-IDF model with a random forest, the strongest published lexical baseline. Because that baseline was reported on a single chronological split (train first 70%, test last 15%; a hard, low-defect tail with $\approx 7.5\%$ positives), we reproduce that *exact* split and test set for a like-for-like comparison, using a matched classifier so any difference comes from the features and not the learner; these single-split numbers are reported separately from, and never conflated with, the fully-online results. As a sanity check our JIT-metrics model reproduces the baseline logistic regression on that split to three decimals,

confirming the protocols are aligned. For the CSTG internal ablation specifically, we additionally report a *same-learner* (matched random forest) plain-TF-IDF row so the “does the CSTG representation beat a bag-of-words” comparison is under one learner rather than mixing linear and non-linear models.

1.9 Scalability, complexity, and statistical-significance setup

Beyond accuracy, the paper characterises the system’s cost and the statistical reliability of its comparisons. All of this is computed *without rebuilding* the knowledge graph: final sizes and growth come from read-only queries against the built graph and from attributing delta edges to their source commits in online order; prediction latency is measured by replaying the cached graph family in memory; and per-commit build/modify cost is obtained by an instrumented replay of the real parse-and-diff work on a stratified sample of commits, cross-checked against an analytical operation-count proxy.

- **Statistical profile.** Per-layer node/edge counts, type and token vocabularies, per-file size distributions (with Gini and skew), and a discrete power-law fit of the per-commit change-size distribution (heavy-tailed, exponent $2 < \alpha < 3$ on every layer).
- **Per-commit growth and modification.** Cumulative nodes/edges over the commit stream and over calendar time (overlaid on the drifting bug rate), the per-commit change-size distribution, and the buggy-vs-benign change-size separation — which is large and highly significant (Mann–Whitney $p \ll 10^{-100}$ on every layer; buggy commits make on the order of $2\text{--}3\times$ larger structural changes), the basic reason the graph is predictive.
- **Time and space complexity.** Build/modify cost is shown to be near-linear in file size ($R^2 \approx 0.99$), i.e. per-commit work is $O(\text{change})$ and space is $O(\text{alive nodes} + \text{retained history})$; prediction latency scales with graph density, and the deployed graph-native fusion predicts at sub-millisecond per commit (thousands of commits/second), so the whole pipeline is comfortably real-time and GPU-free. Each per-commit method’s cost is annotated with its known asymptotic complexity and validated against the measured block-time-vs-graph-size growth.
- **Statistical significance.** Headline comparisons are tested on paired per-commit out-of-sample scores: DeLong’s test for ROC-AUC differences (Holm-corrected within each family of comparisons), paired bootstrap tests and 95% confidence intervals for PR-AUC/MCC/G-Mean, and Cliff’s δ effect sizes. This lets us state which accuracy gaps are significant (e.g. graph richness significantly helps the informative methods; PPR is significantly the best method on the final graph; the semantic channel $F+G$ significantly improves every headline metric over F) and which are within noise.

1.10 Implementation and reproducibility

The graph store is Neo4j (final on-disk footprint ≈ 3.3 GB; working footprint ≈ 5.6 GB with transaction logs). Java is parsed with a single deterministic parser (`javalang`) for the whole pipeline, in killable subprocesses with a watchdog so a pathological file cannot stall the build; a persistent parser worker and an in-memory before-graph cache keep the online build tractable, and a range index on each node label’s `id` keeps per-commit cost constant as the graph grows (an absent index was, at one point, the dominant cost — turning an $O(\text{change})$ build into an accidentally quadratic one). All inference code is CPU-only Python (scikit-learn linear models, custom sparse graph routines, gensim/PyKEEN for the embeddings where used); no GNN and no GPU are required. Feature streams and per-method scores are cached to disk so every experiment, table, figure, and statistical test can be regenerated without a live database or a graph rebuild, and companion Jupyter notebooks reproduce all figures from those caches. Randomised components

(embedding initialisation, bootstrap resampling) use fixed seeds. Determinism of node identity across versions (the same source always yields the same parse-local ids) is what makes the online build itself reproducible.

Summary of the final configuration. The deployed predictor is a fully-online, GPU-free, multimodal commit knowledge graph on `apache/groovy` (8,059 ApacheJIT commits, $\approx 20\%$ buggy, 2003–2019): a Core process layer, an AST delta layer, and the CSTG semantic-text layer, over which the fusion $F+G = \text{RN}+\text{PPR}+\text{CSTG}$ predicts each commit prequentially from strictly-past state, warm-up 40% (3,223 commits), block 200, evaluated on the remaining 4,836 commits with a seven-metric plus effort-aware suite and paired significance testing.

2 Environment, Software Versions, and Model-Configuration Checklist

For completeness and reproducibility, this section answers a standard checklist of setup details — operating system and hardware, software and library versions, training hyper-parameters, model architecture, and stopping/tuning. Several checklist items are framed for *deep-learning* pipelines (optimiser, learning rate, batch size, epochs, hidden-layer dimensions, early stopping). Our final methodology is deliberately **GPU-free and GNN-free**: the inference models are classical graph algorithms (random-walk / label-propagation / matrix-factorisation) with *linear* (logistic-regression) heads, plus one compact bilinear knowledge-graph embedding trained by a short custom SGD. Where a checklist item has no analogue in this design, we say so explicitly and explain why, rather than inventing a value.

2.1 Operating system and hardware

All building and all experiments were run on a single commodity laptop, with no GPU and no cluster:

- **Operating system:** Microsoft Windows 11 (64-bit, build 26,200). The pipeline is OS-agnostic (pure Python plus a Dockerised Neo4j) and also runs on Linux/macOS; the Neo4j administration helpers assume a Docker-hosted server.
- **CPU:** Intel Core i7-13620H (13th generation), 10 physical cores / 16 logical threads, 2.4 GHz base clock. **No GPU is used at any stage** (build, inference, or analysis).
- **Memory (RAM):** 16 GB. The whole pipeline is designed to run within this budget: the online build stores only one evolving structure per file (not a snapshot per commit), and the inference/analysis code streams and caches to disk, so no stage requires loading the full graph into memory at once.
- **Graph store:** Neo4j runs in a Docker container; the built Groovy graph occupies ≈ 3.3 GB on disk (≈ 5.6 GB with transaction logs).

2.2 Software and library versions

The exact versions used are listed in Table 2. The final pipeline depends only on the mainstream scientific-Python stack, a Java parser, and a Neo4j driver; there is **no deep-learning framework** (no PyTorch, TensorFlow, or Keras) anywhere on the critical path, consistent with the CPU-only design.

A note on gensim / PyKEEN. Earlier iterations experimented with `gensim` (node2vec/DeepWalk) and `PyKEEN` (TransE/DistMult). These are **not** used by the final methodology: the deployed DeepWalk embedding (DW) is a self-contained truncated-SVD of a PPMI matrix, and the deployed knowledge-graph embedding (KGE) is our own compact DistMult in NumPy (below). No external embedding library is on the final critical path.

Table 2: Software and library versions used for all builds and experiments.

Component	Version	Role
Python	3.13.5	runtime for the entire pipeline
Neo4j	2026.05 (Community)	graph store (Dockerised)
neo4j (driver)	6.2.0	Python→Neo4j Bolt driver
git	2.45.1	reads per-commit file blobs / diffs
javalang	0.13.0	Java parser (AST/CFG/DFG/PDG/SEQ builders)
NumPy	2.2.6	arrays; the graph-method kernels
SciPy	1.16.2	sparse matrices; statistics (Mann–Whitney, ranks)
scikit-learn	1.7.2	logistic-regression heads, TF-IDF, RF/GB baselines
pandas	2.3.2	CSV / tabular handling
networkx	3.5	in-memory graph construction in the builders
matplotlib	3.10.6	figures
nbformat / nbconvert	5.10.4 / 7.17.1	build and execute the result notebooks
PyYAML	6.0.2	per-project configuration (reproducibility package)

2.3 Training hyper-parameters

The checklist items *optimiser*, *learning rate*, *batch size*, and *number of epochs* are properties of gradient-descent neural training. Our final models are not neural, so most of these **do not apply**; the few genuine training knobs are given precisely.

Optimiser. *Mostly not applicable.* The linear heads on every graph method and on the M and G fusion channels are **balanced logistic regressions solved by L-BFGS** (scikit-learn, `solver="lbfgs"`, `class_weight="balanced"`, `max_iter= 1000–2000`); L-BFGS is a quasi-Newton batch solver, so there is no stochastic optimiser, no learning-rate schedule, and no mini-batching. The *only* stochastic-gradient component in the whole system is the DistMult KGE, trained with plain SGD.

Learning rate. *Only the KGE has one:* the DistMult embedding uses a fixed SGD learning rate of $\eta = 0.05$. All logistic-regression heads are L-BFGS and have no learning rate. DeepWalk (DW) is closed-form (SVD) and has none.

Batch size. *Not applicable in the neural sense.* There is no mini-batch training. The one gradient method (KGE) processes its past triples in internal chunks of 2,048 for vectorised updates, but this is an implementation detail of a full-pass SGD, not a tuned batch-size hyper-parameter. The evaluation protocol does have a *block* granularity (200 commits) at which models are refreshed, but that is a streaming-protocol constant (§1.5), not a training batch size.

Number of epochs. *Only the KGE has epochs:* the DistMult SGD runs 3 epochs over the strictly-past triples at each refit; negative sampling uses 3 corrupted tails per positive. No other model has an epoch count: the logistic heads run to L-BFGS convergence (capped at `max_iter`), and DW/RN/PPR/LP are single-pass or fixed-iteration (below), not epoch-based.

The fixed-iteration counts of the non-embedding graph methods (which play the role “epochs” would in a neural model) are: **PPR** — 40 power-iterations of random-walk-with-restart, restart probability $\alpha = 0.15$, run once per class seed set per block; **LP** — 3 propagation steps with clamping on known past labels; **RN** — a single one-hop weighted vote (no iteration). These are held fixed across all projects and all graphs.

2.4 Model architecture

The checklist item *hidden-layer dimensions* presupposes a multi-layer neural network. Our models have **no hidden layers** — there is no multi-layer network, and specifically **no GNN** (a heterogeneous GNN is explicitly scoped as future work, not part of this study). The only learned latent dimensionalities are the two shallow embedding sizes already reported: **DeepWalk (DW)** = 64 and **DistMult KGE** = 32. Beyond those:

- **RN, PPR, LP** produce a scalar risk score directly from the commit–hub incidence / transition matrix; they have no latent layer at all.
- **DW** embeds commits into $\mathbb{R}^{64}$ by truncated-SVD of the PPMI of the commit–hub co-occurrence (Levy–Goldberg form), then a logistic head; there is no hidden layer, only this one factorisation dimension.
- **KGE (DistMult)** embeds hubs and relations into $\mathbb{R}^{32}$ with a bilinear score; commits are folded in as the mean of their hubs’ embeddings, then a logistic head. Again a single latent dimension, no hidden layers.
- **Text/structural feature widths** (not “hidden” layers, but the input dimensionalities worth recording): the structural TF–IDF and the CSTG text stream are hashed to 2^{18} features (a fixed, stateless hashing vectoriser); the CSTG typed-mass vector is 5-dimensional; the JIT-metric channel is 12-dimensional.

So the “architecture” of the deployed model is: five shallow graph-inference scorers (two of them with a 64- or 32-dim embedding) whose scalar outputs are combined by a single balanced logistic-regression stacker — no hidden layers, no GNN.

2.5 Stopping and tuning

Early-stopping criteria. *Not used, and not applicable.* Because the models are non-neural, there is no validation-loss-based early stopping. The KGE SGD runs a fixed 3 epochs; the logistic heads stop at L-BFGS convergence (or the `max_iter` cap); the fixed-iteration methods (PPR 40, LP 3) run to their set iteration count. The streaming protocol itself has no “stopping”: every post-warm-up commit is scored, and the run ends only when the commit stream ends.

Hyper-parameter tuning strategy. We deliberately **do not tune the model hyper-parameters per project or per graph**, and this is a design choice, not an omission: the whole study is a *controlled comparison*, so the protocol constants (warm-up 40%, block 200, roll 800, refit every 5 blocks), the embedding sizes (64/32), the iteration counts (PPR 40, LP 3, KGE 3 epochs / $\eta = 0.05$ / 3 negatives), and the classifier family (balanced L-BFGS logistic regression) are **held fixed across every representation, method, fusion, and project**. Fixing them is what makes cross-configuration and cross-project differences attributable to the factor under study rather than to per-cell tuning. The one thing that *is* selected — the deployed fusion formula — is chosen not by a hyper-parameter search but by the transparent, reported combinatorial rule of §1.7 (evaluate all $2^5 - 1$ method combinations, take the smallest within a fixed tolerance of the best balanced score). The threshold at which hard decisions are read is likewise not a tuned constant but is re-derived *online* from past-only data every 150 commits (§1.5). The default hyper-parameter values were set once, up front, from standard choices in the literature (e.g. $\alpha = 0.15$ for PPR) and small sanity checks; there was no automated grid/random/Bayesian search over them, precisely to avoid over-fitting the evaluation.

Cross-project note. Every constant above is identical across all subject projects (Groovy and the additional Apache projects processed with the reproducibility package); only the data changes. The single genuinely per-project quantity is the base commit that seeds the structural layers (§1.1), which is a property of each repository’s history, not a tuned hyper-parameter.